Optimizing Execution Plans in a Multistore

Chiara Forresi, Matteo Francia, Enrico Gallinucci(✉), and Matteo Golfarelli

University of Bologna, Cesena, Italy
{chiara.forresi,m.francia,enrico.gallinucci,matteo.golfarelli}@unibo.it

Abstract. Multistores are data management systems that enable query processing across different database management systems (DBMSs); besides the distribution of data, complexity factors like schema heterogeneity and data replication must be resolved through integration and data fusion activities. In a recent work [2], we have proposed a multistore solution that relies on a dataspace to provide the user with an integrated view of the available data and enables the formulation and execution of GPSJ (generalized projection, selection and join) queries. In this paper, we propose a technique to optimize the execution of GPSJ queries by finding the most efficient execution plan on the multistore. In particular, we devise three different strategies to carry out joins and data fusion, and we build a cost model to enable the evaluation of different execution plans. Through the experimental evaluation, we are able to profile the suitability of each strategy to different multistore configurations, thus validating our multi-strategy approach and motivating further research on this topic.

Keywords: Multistore · NoSQL · Join optimization · Cost model

1 Introduction

The decline of the *one-size-fits-all* paradigm has pushed researchers and practitioners towards the idea of *polyglot persistence* [19], where a multitude of database management systems (DBMSs) are employed to support data storage and querying. The motivations are manifold, including the exploitation of the strongest features of each system, the off-loading of historical data to cheaper database systems, and the adoption of different storage solutions by different branches of the same company. This trend has also influenced the discipline of data science, as analysts are being steered away from traditional data warehousing and towards a more flexible and lightweight approach to data analysis. Multistore contexts are characterized by 1) the replication of data across different storage systems (i.e., there is no clean horizontal partitioning) with possibly conflicting records (e.g., the same customer with a different country of residence in different databases), and 2) a high level of schema heterogeneity: records of the same real-world entity may be represented with different schema structures, using different naming conventions for the same information and storing different

© Springer Nature Switzerland AG 2021
L. Bellatreche et al. (Eds.): ADBIS 2021, LNCS 12843, pp. 136–151, 2021.
https://doi.org/10.1007/978-3-030-82472-3_11

information in different databases. The large volume and the frequent evolution of these data hinder the adoption of a traditional integration approach.

In a recent work [2] we have proposed a multistore solution that relies on a dataspace to provide the user with an integrated view of the data. A *dataspace* is a lightweight integration approach providing basic query expressiveness on a variety of data sources, bypassing the complexity of traditional integration approaches and possibly returning best-effort or approximate answers [9]. The dataspace is built in accordance with a *pay-as-you-go* philosophy, i.e., by applying simple matching rules to recognize relationships between data structures and by letting the users progressively refine the dataspace as new relationships are discovered [13]. Users exploit the dataspace to formulate GPSJ (generalized projection, selection and join [12]) queries, i.e., the most common class of queries in OLAP applications. Queries are translated into execution plans that consist of many local computations (to be demanded to the single DBMSs) and a global computation (carried out by the middleware layer).

In this paper, we propose a technique to optimize the execution of GPSJ queries by finding the most efficient execution plan on the multistore. The main challenge lies in devising a cross-DBMS execution plan that couples data fusion operations (to reconcile replicated data) with the resolution of schema heterogeneity and efficiently provides a correct result. In particular, the paper provides the following contributions: (1) the extension of the multistore scenario presented in [2] to consider replicated data across different databases; (2) the introduction of three join strategies that define alternative query execution plans by relying on different schema representations; (3) the presentation of a multi-DBMS cost model to compare the complexity of execution plans in terms of disk I/O and choose the most efficient one; (4) the evaluation of the proposed join strategies on a representative example of two real-world entities (i.e., customers and orders) modeled with different schemas in different databases and linked by a many-to-one relationship.[1]

The paper is structured as follows. In Sect. 2 we introduce the background knowledge on our multistore; in Sect. 3 we introduce the join strategies and in Sect. 4 we present the cost model; the evaluation of the join strategies based on the cost model is given in Sect. 5. Section 6 discusses related work, and Sect. 7 draws the conclusions.

2 Multistore Preliminaries

In this section we provide the preliminary concepts to understand the multistore scenario; we refer the reader to [2] for further details. Figure 1 shows a functional overview of our prototypical implementation. The user interacts with the multistore by submitting GPSJ queries to the Query planner through a custom API service; the planner translates these queries into execution plans, which are

[1] Remarkably, many-to-one relationships are at the base of the multidimensional model and GPSJ queries [12], as well as our dataspace-based approach [2].

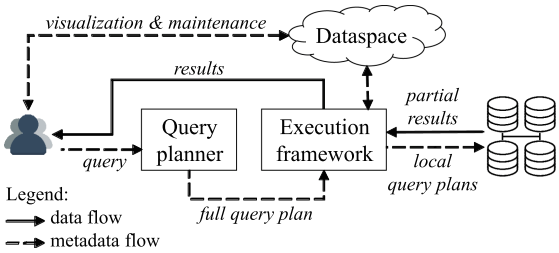

Fig. 1. Overview of our multistore.

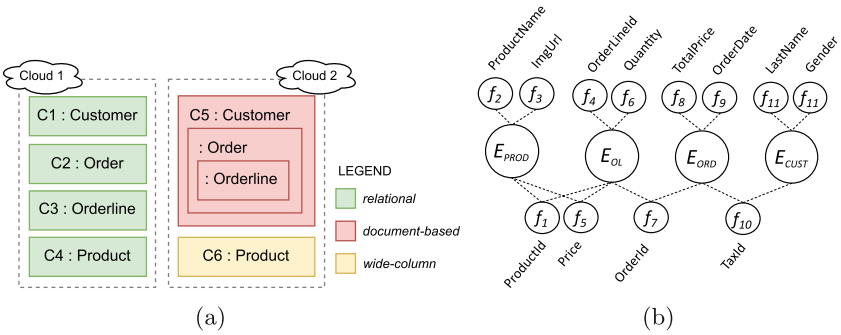

Fig. 2. Overview of our case study and the related dataspace.

formulated onto the Execution framework (i.e., Apache Spark in our implementation). Execution plans are composed of one or more local plans (to be submitted to the underlying DBMSs) and a global plan, that is run in the execution framework; the results are finally returned to the user. The case study is shown in Fig. 2a: it is a Unibench-based [22] multi-cloud scenario, where two branches of the same company store overlapping records of the same entities (i.e., customers, orders, orderlines, and products) on different DBMSs. The first branch relies on an RDBMS, the other on a document store and a wide-column store.

2.1 Basic Concepts

NoSQL DBMSs embrace a *soft-schema* approach that allows collections to contain records with different schemas. In [2] we describe how the multistore is able to manage schema heterogeneity; given that intra-collection schema heterogeneity does not affect execution plan optimization, we assume schema homogeneity among the records of the same collection. Here, we introduce a schema definition that provides a view of the records in first normal form by hiding the denormalization due to the nesting of records and exposing the relationships between schemas at different nesting levels.

Definition 1 (Schema). *A schema* $S = \{a_1, \ldots, a_{|S|}\}$ *is a set of attributes that applies to one or more records in a collection* C*. The attribute that uniquely*

```
{"cid": "C001",
 "firstName": "Alice",
 "address": {
   "street": "1, First St."
 },
 "orders": [{
     "oid": "O001",
     "date": "2020-01-01",
     "price": 1552.59
 }] }
```

Fig. 3. A sample JSON document corresponding to two records.

identifies the records with schema S is the key*, defined as $key(S)$. We use S^{μ} to denote the list of array attributes in C that must be unnested to unveil the records of S (if any).*

For the sake of simplicity, we assume all keys to be simple. Given a record r, its schema (denoted with S_r) is the set of attributes directly available in r (i.e., those in the top level). If r is contained within the array attribute a of record r', then (i) S_r also includes $key(S_{r'})$ (this is necessary to maintain the relationship between the schema of a nested record and the one of the parent record), and (ii) S_r^{μ} is built by extending $S_{r'}^{\mu}$ with a as the last attribute to be unnested.

Example 1. The JSON document in Fig. 3 is composed of two records, *cust* (in blue) and *ord* (in green; this is nested in *cust*), with the following schemas (keys are underlined): $S_{cust} = \{\underline{\mathsf{cid}}, \mathsf{firstName}, \mathsf{address.street}, \mathsf{orders}\}$, with $S_{cust}^{\mu} = []$; $S_{ord} = \{\mathsf{cid}, \underline{\mathsf{orders.oid}}, \mathsf{orders.date}, \mathsf{orders.price}\}$, with $S_{ord}^{\mu} = [\mathsf{orders}]$.

2.2 Dataspace Modeling

The dataspace is defined by a set of abstract concepts named *features* and *entities*: the former provides a unique representation of semantically equivalent attributes, while the latter provides a representation of real-world entities. The composition of features and entities fundamentally relies on the recognition of relationships between attributes and schemas, which is done through mappings.

Definition 2 (Mapping). *A mapping m is a pair $m = (a_i, a_j)$ that expresses a semantic equivalence between two primitive attributes a_i and a_j. The existence of a mapping between a_i and a_j is indicated with $a_i \equiv a_j$.*

Mappings are specified between attributes of different schemas and they reveal the relationship between such schemas. Consider two schemas S_i and S_j. If $key(S_i) \equiv key(S_j)$, then we can infer a *one-to-one* relationship, represented as $S_i \leftrightarrow S_j$. If $a_k \equiv key(S_j) : a_k \in \{S_i \backslash key(S_i)\}$, then we can infer a *many-to-one* relationship from S_i to S_j, represented as $S_i \xrightarrow{a_k} S_j$. If $a_k \equiv a_l : a_k \in \{S_i \backslash key(S_i)\}, a_l \in \{S_j \backslash key(S_j)\}$, no direct relationship exists between the two schemas; *many-to-many* relationships are not considered.

Example 2. Consider the following two schemas, $S_1 = \{\underline{\mathsf{cid}}, \mathsf{firstName}\}$ modeling records of customers, and $S_2 = \{\underline{\mathsf{oid}}, \mathsf{orderDate}, \mathsf{custId}, \mathsf{custName}\}$ modeling records of orders. Two mappings can be defined on such schemas, i.e.,

$m_1 = (\underline{\mathsf{cid}}, \mathsf{custId})$, and $m_2 = (\mathsf{firstName}, \mathsf{custName})$. Since $key(S_1)$ is mapped to a non-key attribute of S_2, we can infer a many-to-one relationship between the two schemas. Mapping m_2 does not reveal any direct relationship, but it shows that S_2 is denormalized as it models customer information at the order level.

Mappings recognize that there is a semantic equivalence between two attributes in different schemas, thus we need to address all of them through a unique reference. This is the purpose of features.

Definition 3 (Feature). *A feature represents either a single attribute or a group of attributes mapped to each other. We define a feature as* $f = (a, M, \wedge\!\!\!\backslash)$*, where* a *is the* representative *attribute of the feature;* M *is the set of mappings that link all the feature's attributes to the representative* a*;* $\wedge\!\!\!\backslash : (v_i, v_j) \rightarrow v_k$ *is an associative and commutative function that resolves the possible conflicts between the values of any two attributes* (a_i, a_j) *belonging to* f *and returns a single value* v_k*.*

Let $attr(f)$ be the set of attributes represented by f (i.e., the representative attribute plus those derived from the mappings). Given a record r, the conflict resolution function $\wedge\!\!\!\backslash$ can be applied to $r[a_i]$ and $r[a_j]$ if $\{a_i, a_j\} \subseteq attr(f)$; we refer the reader to [4] for an indication about different methods to define conflict resolution functions. Also, we remark that an attribute is always represented by one and only one feature; thus, for any two features f_i and f_j, it is $attr(f_i) \cap attr(f_j) = \varnothing$. We use $rep(f)$ to refer to the representative attribute of f, and $rep(a)$ as short for $rep(feat(a))$. As features are used to represent semantically equivalent attributes, we introduce the concept of *entity* to represent real-world entities (e.g. customers, orders).

Definition 4 (Entity). *An entity* E *is a representation of a real-world entity. It is identified by a feature that acts as a key in at least one schema; we refer to such feature as* $key(E)$*. Also, we use* $feat(E)$ *to refer to the set features whose attributes describe a property of* E*.*

For simplicity, we assume that all instances of the same entity are represented by same key feature. In particular, entities are obtained by detecting the features that act as a key in at least one schema. We can infer a many-to-one relationship from E_i to E_j based on a feature f if $key(E_j) = f$ and $f \in feat(E_i), f \neq key(E_i)$; we represent this as $E_i \xrightarrow{f} E_j$. Ultimately, the dataspace simply consists of the obtained entities and features. The dataspace corresponding to our case study is shown in Fig. 2b.

The dataspace is built semi-automatically in accordance with a pay-as-you-go philosophy: schemas are automatically extracted, while mappings are inferred by applying simple matching rules. Mappings are then used to recognize relationships between the schemas and to infer features and entities. At any time, the users may refine the dataspace as new relationships are discovered. Ultimately, the dataspace is exploited to formulate GPSJ queries, i.e., the most common class of queries in OLAP applications; a typical OLAP query consists of a group-by

set (i.e., the features used to carry out an aggregation), one or more numerical features to be aggregated by some function (e.g., sum, average), and (possibly) some selection predicates. We refer the reader to [2] for details on the process to obtain the dataspace and to translate a query formulated on the dataspace into an execution plan.

2.3 Data Fusion Operations

Given the expressiveness of GPSJ queries, the execution plan of a query is formulated in the Nested Relational Algebra (NRA). Here, we slightly extend NRA to handle some operations required by our multistore scenario.

The most important addition to NRA is the extension of the join operator's semantics to handle data fusion. We do this by introducing an operator called *merge* ($\sqcup$) , i.e., an adaptation to our scenario of the *full outer join-merge* operator introduced in [18] to address the extensional and intensional overlap between schemas. In particular, the records belonging to the same entity (e.g., customers) can be partially overlapped, both in terms of instances (e.g., the same customer can be repeated across different schemas) and in terms of schemas (e.g., the name of the customer can be an attribute of two different schemas). We aim to keep as much information as possible when joining the records of two schemas, both from the extensional and the intensional points of view. The merge operator ($\sqcup$) answers this need by (i) avoiding any loss of records, (ii) resolving mappings by providing output in terms of features instead of attributes, and (iii) resolving conflicts whenever necessary. We define the merge operator as follows.

Definition 5 (Merge operation). *Let R_i and R_j be the recordsets of two schemas S_i and S_j, and consider $(a_k, a_l) \in (S_i, S_j)$ such that $a_k \equiv a_l$, i.e., $\exists f : \{a_k, a_l\} \subseteq attr(f)$. The merge of the two recordsets $R_i \sqcup_f R_j$ produces a recordset R_{ij} with schema $S_{ij} = S_i^* \cup S_j^* \cup S_{ij}^{\cap}$ such that:*

- $S_i^* = \{a \in S_i : \nexists\ a' \in S_j\ s.t.\ a \equiv a'\}$
- $S_j^* = \{a' \in S_j : \nexists\ a \in S_i\ s.t.\ a \equiv a'\}$
- $S_{ij}^{\cap} = \{rep(a) : a \in S_i, \exists a' \in S_j\ s.t.\ a \equiv a'\}$

R_{ij} results in a full-outer join between R_i and R_j where the values of attributes linked by a mapping are merged through function $\wedge\!\!\!\backslash$. In particular, given a record $r \in R_{ij}$ obtained by joining $s \in R_i$ and $t \in R_j$ (i.e., $s[a_i] = t[a_j]$), it is $r[rep(a)] = \wedge\!\!\!\backslash(s[a], t[a'])\ \forall\ (a, a')\ s.t.\ a \in S_i, a' \in S_j, a \equiv a'$.

Example 3. Let $R_1 \sqcup_{f_{id}} R_2$, $s \in R_1$ and $t \in R_2$ two records with schema S_1 and S_2 respectively, $S_1 = \{a_1, a_3\}$, $S_2 = \{a_2, a_4\}$, $f_{id} = \{a_1, a_2\}$, $f_{name} = \{a_3, a_4\}$. Let $s[a_1] = t[a_2]$, $s[a_3] =$ "Smith" and $t[a_4] =$ "Smiht". The merge of s and t produces a record r where $r[a_3] = \wedge\!\!\!\backslash(s[a_3], t[a_4])$ and $\wedge\!\!\!\backslash$ is the a conflict resolution function that decides between "Smith" and "Smiht".

The merge operation is defined between recordsets, thus it is also applicable between the records of two arrays within the same collection. Consider a collection of customers with schema $S = \{\underline{\mathsf{cid}}, \mathsf{o1}, \mathsf{o2}\}$, where o1 and o1 are arrays of

C^{Ne} (nested schema)

cid	firstName	address.street	orders
C001	Alice	1, First St.	oid / date / Price O001 / 01/01/2020 / 10.00 O002 / 01/02/2020 / 20.00
C002	Bob	2, Second Av.	oid / date / Price O003 / 02/01/2020 / 25.00

C^{NY} (normalized customer schema)

cid	firstName	address.street
C001	Alice	1, First St.
C002	Bob	2, Second Av.

C^{NX} (normalized order schema)

cid	oid	date	price
C001	O001	01/01/2020	10.00
C001	O002	01/02/2020	20.00
C002	O003	02/01/2020	25.00

C^{Fl} (flat schema)

cid	firstName	address.street	oid	date	price
C001	Alice	1, First St.	O001	01/01/2020	10.00
C001	Alyce	1, First St.	O002	01/02/2020	20.00
C002	Bob	2, Second Av.	O003	02/01/2020	25.00

Fig. 4. Three different schema representations of entities customer and orders.

orders. The merge operation is applicable between o1 and o2 to merge the records of orders. In the context of our multistore, this operation is always carried out in combination with the unnest of the merged array. Therefore, we introduce the *simultaneous unnest* operator $\overline{\mu}$ to carry out both operations, i.e., merging two (or more) array attributes and unnesting the result.

Definition 6 (Simultaneous unnest operation). *Let R_1 be a recordset with schema $S_1 = \{a_k, a_l, a_m\}$ where a_k is a primitive attribute, a_l and a_m are array attributes; also, let R_2 be the recordset in a_l with schema S_2, R_3 the recordset in a_m with schema S_3, $key(S_2) = key(S_3) = f$. The simultaneous unnest of a_l and a_m, declared as $\overline{\mu}_{[a_l,a_m]}(R_1)$, is translated to $\mu_{a_n}(R_1)$ where a_n is the result of $R_2 \sqcup_f R_3$.*

3 Query Plans and Optimization

In the multistore, the same real-world domain can be modeled in different ways in different DBMSs. The relational model favors normalized schemas (**NoS**), whereas non-relational models encourage different forms of denormalization, mainly nested schemas (**NeS**) and flat schemas (**FlS**). Figure 4 shows an example of these schema representations considering the relationship between customers and orders from our case study.

To define an efficient query plan we must consider that (1) a common schema representation must be chosen to join data, and (2) the query can either optimize data fusion activities or the push-down of operations to the local DBMSs, but not both of them. Indeed, data fusion is favored by normalizing all data (i.e., by choosing NoS as the common representation), whereas pushing operations down to the local DBMSs entails the transformation of data into NeS or FlS (as data in NeS/FlS represent the result of a nest/join operation between data in NoS). Therefore, the choice between the two optimizations is related to the choice of the common representation, each of which produces a different execution plan. The efficiency of the latter ultimately depends on the operations required by the query and on the original schema representation of the data. For this reason, the

Table 1. Schema alignment operations.

From schema	To schema	Operations
NoS (C^{NX}, C^{NY})	FlS (C^{Fl})	$(C^{NX}) \bowtie (C^{NY})$
	NeS (C^{Ne})	$v((C^{NX}) \bowtie (C^{NY}))$
NeS (C^{Ne})	FlS (C^{Fl})	$\mu(C^{Ne})$
	NoS (C^{NX})	$\pi(C^{Ne})$
	NoS (C^{NY})	$\mu(\pi(C^{Ne}))$
FlS (C^{Fl})	NoS (C^{NX})	$\gamma(\pi(C^{Fl}))$
	NoS (C^{NY})	$\pi(C^{Fl})$
	NeS (C^{Ne})	$v(C^{Fl})$

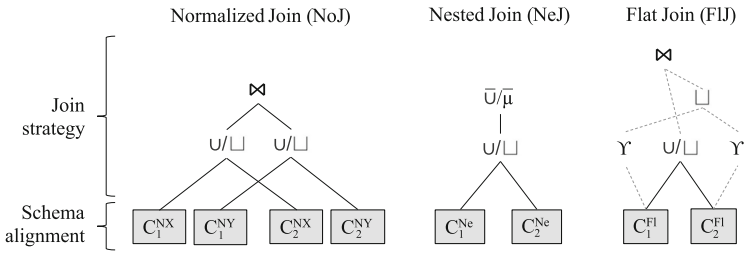

Fig. 5. Execution plan skeleton for each join strategy

query planner produces all three execution plans (one for each common schema representation that may be chosen) and later evaluates the most efficient one through the proposed cost model. Execution plans are composed of two steps.

1. A schema alignment step, where the data in each DBMS is transformed on-the-fly into the chosen reference schema representation.
2. A joining step to carry out the integration and merging of the data, including the data fusion and conflict resolution operations where necessary.

Table 1 shows the NRA operations required to compute a transformation from one schema representation to another (limited to two entities). These operations can be pushed down to the DBMSs if the latter support the former. Then, the strategy to join the aligned data depends on the common schema representation that is chosen. Figure 5 describes the skeleton of the execution plans for each strategy, considering two entities being stored on two DBMSs. Each plan may present some variations depending on the actual need for data fusion between the two entities. In particular, we outline three overlap scenarios: *full* (records of both E_Y and E_X may be replicated in different databases), *partial* (only records of E_Y may be replicated in different databases), or *absent*.

Variations in the execution plans are represented by (i) alternative operations: $\cup$ / $\sqcup$ is used to indicate that simple union $\cup$ is adopted when data fusion is not needed, merge $\sqcup$ otherwise (the same applies to $\overline{\cup}/\overline{\mu}$); (ii) additional oper-

ations: the dotted edges represent paths to optional operations that are carried out only when data fusion is needed. For each join strategy, we first discuss the execution plan in the full overlap scenario and then discuss the variations for the other scenarios.

Normalized Join (NoJ) Strategy. NoJ starts by separately merging the records belonging to the same entity (i.e., C_1^{NX} with C_2^{NX}, and C_1^{NY} with C_2^{NY}) and then joins the obtained results. This strategy follows the footsteps of a traditional integration approach, except that data fusion is carried out at query time. In partial (and no) overlap scenarios, the execution plan is modified by replacing the first merge operation on E_Y (and E_X) with a simple union.

Nested Join (NeJ) Strategy. The two nested collections can be joined directly by following a top-down approach. NeJ starts by merging the collections on the key of E_Y (thus applying data fusion to the upper level of the nested collection); this results in a new collection where the reconciled records of E_Y contain two arrays, each storing E_X's records for the original collections. The subsequent step is to simultaneously unnest these arrays (see Definition 6), in order and apply data fusion to the lower level as well. In the partial overlap scenario, the final simultaneously unnest is replaced by a simple array union. Additionally, in no overlap scenario, the initial merge is also replaced by a simple union.

Flat Join (FlJ) Strategy. Differently from previous strategies, the variations in presence or absence of data fusion need are more significant. In full overlap scenario, the execution plan requires to (i) separately aggregate both collections to extract a normalized view over E_Y; (ii) merge the obtained view to apply data fusion, (iii) join the reconciled records of E_Y with the reconciled records of E_X. The partial overlap scenario differs in the fact that records of E_X must not be reconciled, thus the first merge operation (i.e., the one between C_1^{Fl} and C_2^{Fl}) is replaced by a union. In no overlap scenario, all the optional operations in the execution plan (i.e., those necessary to reconcile E_Y's records) are not necessary, and it suffices to union C_1^{Fl} with C_2^{Fl}.

Remarkably, the efficiency of each strategy depends on the multistore configuration and the selected query. If the query involves data in a single schema representation, the respective join strategy is most likely to be the cheapest, as schema alignment is not necessary. Also, pushing operations (e.g., nest and join) down to the underlying DBMSs favors the usage of statistics and optimization the may not be available at the middleware level; however, by postponing the data fusion operations, the complexity of the execution plan increases (especially for FlJ, as seen above).

4 Cost Model

The identification of the most convenient join strategy to execute a query is demanded to a novel cost model that estimates the cost of our cross-DBMS GPSJ queries. In particular, it estimates the cost of every NRA operation (including the newly defined ones) by measuring the amount of data read and written to

Table 2. Cost model parameters and basic functions

Parameter/function	Description
$NR(C)$	Number of records in C
$NNR(C)$	If C is nested, number of nested records
$Len(C)$	Average byte length of a record in C
DPS	Size of a disk page
$PT(C)$	Number of partitions into which records in C are organized
NB	Number of memory buffers
$NP(C)$	Number of disk pages occupied by C: $\lceil \frac{NR(C) \cdot Len(C)}{DPS} \rceil$
$Part(C, nPart)$	Number of disk pages occupied by one of $nPart$ partitions of C

Table 3. Cost model for NRA operations and additional supporting operations

Operation	Description	Estimated cost	Sup.
$CA(C)$	Collection access	$NP(C)$	RMCS
$\pi(C)$	Projection	$NP(C)$	RMCS
$\gamma(C)$	Aggregation	$Sort(C) + NP(C)$	RM*S
$v(C)$	Nest	$Sort(C) + NP(C)$	RM-S
$\mu(C)$	Unnest	$NP(C)$	RM-S
$\overline{\mu}(C)$	Simult. unnest	$NR(C) \cdot SM(Part(C, \lceil \frac{NNR(C)}{NR(C)} \rceil))$	---S
$\overline{\cup}(C)$	Array union	$NP(C)$	RM-S
$(C_1) \bowtie (C_2)$	Join	$min(NLJ(C, C'),\ SMJ(C, C'),\ HJ(C, C'))$	RM-S
$(C_1) \sqcup (C_2)$	Merge	$min(NLJ(C, C'),\ SMJ(C, C'),\ HJ(C, C'))$	RM-S
$(C_1) \cup (C_2)$	Union	$NP(C) + NP(C')$	RM-S
$Shf(C)$	Data shuffle	$3 \cdot NP(C)$	-M-S
$SM(C)$	Sort-merge	$2 \cdot NP \cdot (\lceil log_{NB-1} NP \rceil + 1)$	RM-S
$Sort(C)$	Data sort (central.)	$SM(C)$	RM--
	Data sort (distrib.)	$Shf(R) + PT(C) \cdot SM(Part(C, PT(C)))$	---S
$HJ(C, C')$	Hybrid hash join	$3 \cdot (NP(C) + NP(C'))$	R---
$NLJ(C, C')$	Nested loops join	$NP(C) + NR(C) \cdot NP(C')$	R---
		$NP(C) + NR(C) \cdot NP(C') + Unnest(C'')$	-M--
$SMJ(C, C')$	Sort-merge join	$Sort(C) + Sort(C') + NP(C) + NP(C')$	R--S

disk (which is the slowest resource and is usually the bottleneck in typical GPSJ queries, even in a distributed framework like Apache Spark [20]). Building a custom cost model is necessary due to the variety of technologies in the multistore: although each execution engine adopts cost-based statistics to estimate query execution times, they are not directly comparable (e.g., MongoDB estimates execution times in milliseconds, while PostgreSQL uses arbitrary time units).

The cost model is focused on the technologies that we adopt in our implementation (see Sect. 2), i.e., RDBMSs in general, MongoDB, Apache Cassandra, and Apache Spark (which is used at the middleware level). To obtain the formulas we relied on existing literature on this subject (e.g., [7,17]) and on the tools' documentation. Table 2 shows the parameters required by the cost model, while Table 3 shows the core part, including the estimated cost for each NRA operation; the lower part of the latter introduces other supporting operations

that are directly or indirectly used in the cost estimate for NRA operations. Table 3 also shows the support of each operation on the adopted technologies, i.e., RDBMSs (R), MongoDB (M), Apache Cassandra (C), and Apache Spark (S); symbol - indicates that the operation is not supported, while symbol * indicates that the operation is partially supported. Table 3 voluntarily omits for space reasons the formulas to estimate the number of records NR of the collection returned by each operation.

Given a query q submitted by the user, the query planner determines an execution plan for each join strategy defined in Sect. 3. Let P be the query plan for q, which can be decomposed into $P_{\mathtt{R}}$, $P_{\mathtt{M}}$, $P_{\mathtt{C}}$ (i.e., the portions of P to be executed on local DBMSs), and $P_{\mathtt{S}}$ (i.e., the portion to be executed on the middleware). The rationale of the decomposition is to push down to the local DBMSs as much computation as possible. Given the complexity of a multistore configuration comprising several technologies and concurrent computations in a distributed environment, we make a number of assumptions to simplify the calculation of $Cost(P)$. In particular, we assume that (i) global computations on the middleware begin after each local computation have been carried out, (ii) each DBMS runs in a non-distributed way on a separate machine, (iii) Apache Spark runs on a certain number of machines $n_{\mathtt{S}}$,[2] and (iv) data is uniformly distributed across Spark's machines. Then, we calculate $Cost(P) = Cost(P_{\mathtt{S}})/n_{\mathtt{S}} + \max_{i \in \mathtt{R,M,C}} Cost(P_{\mathtt{i}})$.

5 Evaluation

The three join strategies are evaluated on a wide range of multistore configurations modeling the representative domain $E_X \rightarrow E_Y$ (where E_Y and E_X represent customers and orders, respectively) by running GPSJ queries that are devised in accordance with a reference model comprising two group-by features, one measure, and no selection predicates (e.g., the average price by product name and customer gender). The configurations vary on the following parameters.

- The number of records for each entity: either 10^2 or 10^5 for E_Y, either $NR_Y \cdot 10$ or $NR_Y \cdot 100$ for E_X.
- The ordering of records in each database: either ordered on $key(Y)$ (i.e., the customer key, which facilitates join and aggregation operations) or not.
- The overlap scenario (see Sect. 3): either absent, partial, or full; to estimate the cardinality we consider an overlap rate of 5% (i.e., 5% of customers on one collection overlap with the customers on the other).

The combination of these parameters determines 3072 physical configurations, which are hard to deploy and evaluate empirically. Thus, the join strategies are evaluated over all configurations by means of our cost model, while the accuracy of the latter is measured by deploying 6 representative configurations

[2] Although the level of parallelism in Spark is given in terms of CPU cores, we consider the number of machines because the cost model is focused on disk IO rather than on CPU computation.

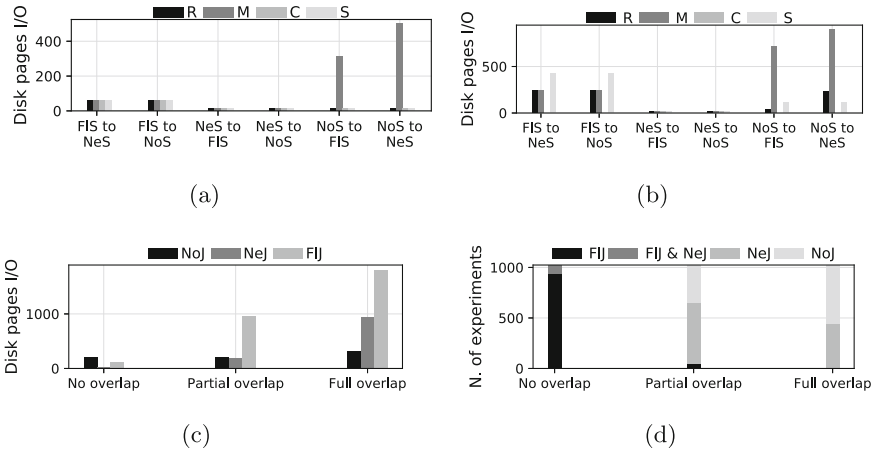

Fig. 6. Estimated cost of schema alignment on different execution engines with collections ordered (a) or not ordered (b) on the join attribute; estimated cost of join strategies (c); number of experiments won by each join strategy (d).

(i.e., all overlap scenarios, either with a low or high gap between the number of records of the two entities) consistently with the cost model assumptions (i.e., centralized DBMSs on separate machines, and Spark running on 4 machines). The results show that, in 83% of the experiments, the cost model identifies the strategy corresponding to the plan with the lowest execution time. With respect to having an oracle that always finds the best execution plan, the choices of our cost model cause an average overhead of 10%; conversely, always adopting a single join strategy returns an average overhead between 77% and 127%.

Single Step Evaluation. The first question we aim to answer is "Given a certain execution engine and a certain schema representation for the collections of E_Y and E_X, what is the cost of aligning both collections to a certain schema representation?". From the results (shown in Figs. 6a and 6b) we learn that NeS is generally the cheapest schema representation to move away from (as projections and unnest operation are the most simple ones) and the most expensive to align to (as nest requires an expensive aggregation operation); conversely, FlS is the most expensive to move away from, as it requires to carry out aggregations. We remark that MongoDB suffers in those transformations that require collections to be joined, due to its limited join capabilities; also, some costs are not estimated for Cassandra due to the lack of support to several operations.

The join strategies define operations that must be carried out in the middleware to consolidate the data obtained from the local DBMSs; therefore we evaluate the costs only with reference to Apache Spark. The results are shown in Fig. 6c. We observe that the cost for NoJ is quite stable over the different overlap scenarios, whereas the cost of NeJ and FlJ is more affected by the latter. As expected, FlJ is the one suffering the most the need to carry out data fusion. An interesting duality emerges by comparing these results with the above ones on

schema alignment: NeS appears as the most expensive one to transform into and often the cheapest one to carry out the join, whereas FlS is exactly the opposite. These considerations confirm that the identification of the best execution plan will be significantly influenced by the characteristics of the dataset (in terms of the adopted schema representations, cardinality, and overlap scenarios).

Full Execution Plan Evaluation. Figure 6d shows, for each overlap scenario, the strategy that returns the minimum cost. The results indicate a clear distinction between the scenario with no overlap and the others. In the first case, FlJ emerges as a clear winner (except for a few cases where the estimated cost is the same for NeJ), indicating that DBMS locality can be fully exploited when there is no need to carry data fusion activities. In the other cases, NeJ and NoJ share an almost equal amount of wins, with NeJ slightly favoring the case of partial overlap (consistently with previous single-step results) and NoJ favoring the case where data fusion is needed for both entities (confirming the discussion made in Sect. 3). The main variable that pushes towards one of the two strategies in these configurations is the number of records in NeS and NoS: the higher the imbalance of data towards a certain schema representation, the lower the cost of the respective join strategy. Ultimately, these results validate the proposal of multiple join strategies (as neither of them is optimal under every circumstance) and motivate further research in this direction.

6 Related Work

The variety in terms of data models responds to different requirements of modern data-intensive applications, but providing transparent querying mechanisms to query large-scale collections on heterogeneous data stores is an active research area [21]. While many proposals answer the need for supporting multiple data models, there is a lack of solutions to address both schema heterogeneity and data fusion in polyglot systems. Remarkably, neglecting these issues may lead to incomplete and/or incorrect results.

A naive approach to solve the problem of querying several data models is to transform all datasets into a reference data model—usually the relational one [8]. This kind of solution leads to the loss of the schemaless flexibility and requires continuous maintenance to support schema evolution. A different approach is proposed by *multimodel* systems, which directly support several data models within the same platform (e.g., OrientDB, ArangoDB). Inter-data model querying is enabled by a custom query language to support nested structures and graph queries. An example is [3], which discusses a data warehousing approach relying on a multimodel system. Differently from our approach, these systems do not directly support the resolution of schema heterogeneity and data fusion—thus requiring manual activities (or additional tools) to clean either the original data or the query results.

In recent years, *multistores* and *polystores* have emerged to provide integrated access and querying to several heterogeneous stores through a mediator layer (middleware) [21]. The difference between multistores and polystores lies in

whether they offer a single or multiple querying interfaces, respectively. Among the most notable are BIGDAWG [10], TATOOINE [6], and CloudMDsQL [14]. These systems vary in the functionalities they support (e.g., the available data models and storage systems, the support to the ingestion process, the expressiveness of the querying language, the possibility to move data from one DBMS to another). However, none of them supports data fusion.

Effectively supporting querying on a heterogeneous system with overlapping records requires the adoption of data fusion techniques [5]. The literature on this subject is very wide, thus we refer the reader to a recent survey [16]. Remarkably, related work in this area does not apply directly to a polyglot system. To the best of our knowledge, the only proposal that considers a scenario requiring data fusion in a polyglot system is QUEPA [15], where the authors present a polystore-based approach to support query augmentation. The approach is meant to complement other polystores that actually support cross-DBMS querying, and record linkage techniques are only used to find related instances in different DBMSs, but not to solve conflicts. Another work that proposes on-the-fly integration and schema heterogeneity resolution in an analytical context is [11]; however, the proposed approach is limited to document-oriented databases and does not consider data fusion.

7 Conclusions

In this work, we have extended our multistore approach [2] with a cost-based optimization of execution plans by devising and evaluating three join strategies in presence of data replication. The results demonstrate that, depending on the configuration of the multistore and the distribution of the data, different join strategies may emerge as the most convenient to follow to structure the query execution plan. This validates the multi-strategy approach and motivates further research in this direction. In future work, the first step will be to apply the join strategies on a wider domain including multiple entities. This step alone opens to several challenges, as the variety of schema representations will considerably increase and may impact significantly on the choice of the best strategy. The cost model can be also improved from many perspectives, by (i) considering the impact of distributed database implementations (other than the middleware execution engine), (ii) adopting finer modeling of concurrent computations, and (iii) integrating the cost model with the existing one for GPSJ queries on Apache Spark [1]. We plan to further evaluate the join strategies over a broader selection of queries, including selection predicates with varying selectivity, which will require additional statistics to be collected from the different DBMSs. Ultimately, we will consider the addition of partial aggregation push-down to the local databases to increase the efficiency of our multistore system.

References

1. Baldacci, L., Golfarelli, M.: A cost model for SPARK SQL. IEEE Trans. Knowl. Data Eng. **31**(5), 819–832 (2019)
2. Ben Hamadou, H., Gallinucci, E., Golfarelli, M.: Answering GPSJ queries in a polystore: a dataspace-based approach. In: Laender, A.H.F., Pernici, B., Lim, E.-P., de Oliveira, J.P.M. (eds.) ER 2019. LNCS, vol. 11788, pp. 189–203. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-33223-5_16
3. Bimonte, S., Gallinucci, E., Marcel, P., Rizzi, S.: Data variety, come as you are in multi-model data warehouses. Inf. Syst. 101734 (2021)
4. Bleiholder, J., Naumann, F.: Declarative data fusion – syntax, semantics, and implementation. In: Eder, J., Haav, H.-M., Kalja, A., Penjam, J. (eds.) ADBIS 2005. LNCS, vol. 3631, pp. 58–73. Springer, Heidelberg (2005). https://doi.org/10.1007/11547686_5
5. Bleiholder, J., Naumann, F.: Data fusion. ACM Comput. Surv. (CSUR) **41**(1), 1–41 (2009)
6. Bonaque, R., et al.: Mixed-instance querying: a lightweight integration architecture for data journalism. Proc. VLDB Endow. **9**(13), 1513–1516 (2016)
7. DeWitt, D.J., et al.: Implementation techniques for main memory database systems. In: Proceedings of the 1984 SIGMOD Annual Meeting, pp. 1–8 (1984)
8. DiScala, M., Abadi, D.J.: Automatic generation of normalized relational schemas from nested key-value data. In: 2016 ACM SIGMOD International Conference on Management of Data, pp. 295–310. ACM (2016)
9. Franklin, M.J., Halevy, A.Y., Maier, D.: From databases to dataspaces: a new abstraction for information management. SIGMOD Rec. **34**(4), 27–33 (2005)
10. Gadepally, V., et al.: The BIGDAWG polystore system and architecture. In: 2016 IEEE High Performance Extreme Computing Conference, pp. 1–6. IEEE (2016)
11. Gallinucci, E., Golfarelli, M., Rizzi, S.: Approximate OLAP of document-oriented databases: a variety-aware approach. Inf. Syst. **85**, 114–130 (2019)
12. Golfarelli, M., Maio, D., Rizzi, S.: The dimensional fact model: a conceptual model for data warehouses. Int. J. Coop. Inf. Syst. **7**(2–3), 215–247 (1998)
13. Jeffery, S.R., Franklin, M.J., Halevy, A.Y.: Pay-as-you-go user feedback for dataspace systems. In: 2008 ACM SIGMOD International Conference on Management of Data, pp. 847–860. ACM (2008)
14. Kolev, B., et al.: CloudMDSQL: querying heterogeneous cloud data stores with a common language. Distrib. Parallel Databases **34**(4), 463–503 (2016)
15. Maccioni, A., Torlone, R.: Augmented access for querying and exploring a polystore. In: 34th IEEE International Conference on Data Engineering, ICDE 2018, pp. 77–88. IEEE Computer Society (2018)
16. Mandreoli, F., Montangero, M.: Dealing with data heterogeneity in a data fusion perspective: models, methodologies, and algorithms. In: Data Handling in Science and Technology, vol. 31, pp. 235–270. Elsevier (2019)
17. Mishra, P., Eich, M.H.: Join processing in relational databases. ACM Comput. Surv. **24**(1), 63–113 (1992)
18. Naumann, F., Freytag, J.C., Leser, U.: Completeness of integrated information sources. Inf. Syst. **29**(7), 583–615 (2004)
19. Sadalage, P.J., Fowler, M.: NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Pearson Education, London (2013)
20. Shi, J., et al.: Clash of the titans: mapreduce vs. spark for large scale data analytics. Proc. VLDB Endow. **8**(13), 2110–2121 (2015)

21. Tan, R., Chirkova, R., Gadepally, V., Mattson, T.G.: Enabling query processing across heterogeneous data models: a survey. In: 2017 IEEE International Conference on Big Data, pp. 3211–3220. IEEE Computer Society (2017)
22. Zhang, C., Lu, J., Xu, P., Chen, Y.: UniBench: a benchmark for multi-model database management systems. In: Nambiar, R., Poess, M. (eds.) TPCTC 2018. LNCS, vol. 11135, pp. 7–23. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-11404-6_2